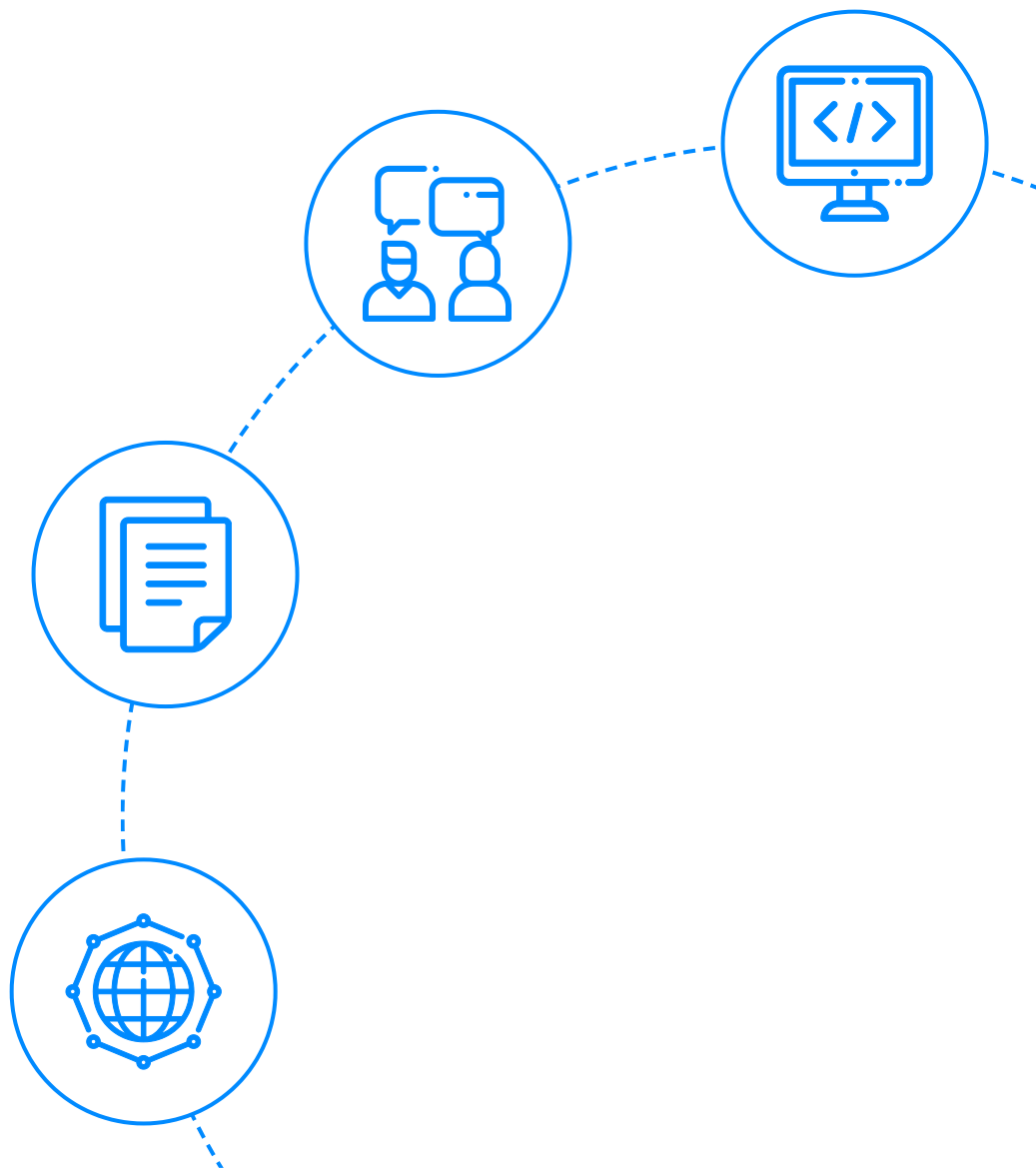




InterviewBit

Hive Interview Questions



To view the live version of the page, [click here](#).

© Copyright by Interviewbit

Contents

Hive Interview Questions for Freshers

1. What applications are supported by Hive?
2. What are the different types of tables available in Hive?
3. What is the difference between managed and external tables?
4. Where does the data of a Hive table get stored?
5. Can Hive be used in OLTP systems?
6. Can a table name be changed in Hive?
7. Can the default location of a managed table be changed in Hive?
8. What is a Hive Metastore?
9. What are the types of Metastore?
10. What is the difference between Local and Remote Metastore?
11. What is the default Apache Hive metastore database?
12. Can multiple users use one metastore?
13. What are the three different modes in which Hive can be operated?
14. Is there a data type in Hive to store date information?
15. Why is partitioning used in Hive?
16. What is dynamic partitioning and when is it used in Hive?
17. What are the Hive collection data types?
18. Is it possible to run UNIX shell commands in Hive?
19. Is it possible to execute Hive queries from a script file?
20. What is a .hiverc file?

Hive Interview Questions for Freshers

(.....Continued)

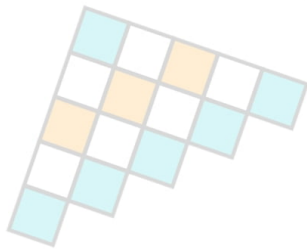
21. Are multi-line comments supported by Hive?
22. How can you view the indexes of a Hive table?
23. What is the Hive ObjectInspector function?
24. What is Bucketing in Hive?
25. How is bucketing helpful?
26. Can you specify the name of the table creator in Hive?
27. What is HCatalog?
28. What is UDF in Hive?
29. What does `/*streamtable(table_name)*/` do?

Hive Interview Questions for Experienced

30. What are the key features of Hive that differentiate it from other big data processing tools?
31. Can you walk me through the Hive query optimization process and the techniques you use to optimize queries?
32. What is the difference between partitioning and bucketing in Hive? When would you choose one over the other?
33. Can you give an example of a complex Hive query you have written and the problem it solved?
34. How do you handle data skewness in Hive? How have you used Hive in conjunction with other big data tools like Spark or Hadoop?
35. Can you explain how dynamic partitioning works in Hive?
36. Have you worked with Hive UDFs (user-defined functions) or UDAFs (user-defined aggregate functions)?
37. What are some best practices for managing and optimizing Hive metastore performance?
38. Can you explain how Hive authorization and authentication work?

Hive Interview Questions for Experienced (.....Continued)

- 39.** How would you address the issue of creating a single Hive table for multiple small CSV files located in the /input directory of HDFS, without compromising the system's performance, given that using many small files can slow down Hadoop's performance?



InterviewBit

Let's get Started

Introduction

The Apache Software Foundation created the query language Hive, which is used for data warehousing and resembles SQL. Large datasets kept in the [Hadoop Distributed File System](#) (HDFS) or other similar file systems can be managed and queried using the platform it offers.

Hive can handle petabytes of data and is made to be highly scalable. Moreover, it offers a familiar SQL-like syntax, making it accessible to users with prior [SQL knowledge](#). MapReduce jobs are created by Hive and then converted into SQL-like queries that are subsequently performed on a Hadoop cluster. This eliminates the need for consumers to learn difficult MapReduce programming by allowing them to create queries in a language they are already comfortable with and analyze enormous amounts of data.



Hive is in good demand if we see the job trends as it is frequently used by companies to aggregate, summarize, and analyze big datasets in data warehousing and business intelligence applications. It can be used to preprocess and manipulate data before supplying it to machine learning algorithms in applications that use machine learning. Hive offers a robust framework for handling and analyzing massive datasets, and thanks to its familiar SQL-like syntax, it is usable by a variety of users.

This article covers important **Hive Interview Questions and Answers** to familiarize candidates with the concept of Hive, its usefulness, its application and the different terminologies used while working with this query language.

- [Hive Interview Questions for Freshers](#)
- [Hive Interview Questions for Experienced](#)
- [Hive MCQ Questions](#)

Hive Interview Questions for Freshers

1. What applications are supported by Hive?

Apache Hive is an open-source [data warehouse framework](#) for querying and analyzing large datasets stored in distributed storage systems such as Hadoop Distributed File System (HDFS), Amazon S3, and others. There are various applications supported by Hive since it is a warehouse and offers various applications in data querying, analysis, and summarization. Some of them are as follows:

1. **Data Warehousing:** Hive is frequently used for data warehousing applications and enables users to handle enormous datasets and carry out intricate searches on structured data kept in Hadoop.
2. **Business intelligence:** Hive gives business intelligence tools an interface and enables the users to see and examine Hadoop data in real time.
3. **Data Analysis:** Hive supports various forms of data analysis, such as machine learning, statistical analysis, and data mining.
4. **Interfacing:** Hive interfaces with numerous tools and data sources and enables users to quickly access and analyze data from various sources.
5. **Data Processing:** Data processing is made possible by Hive, which offers a framework for ETL (Extract, Transform, Load) operations, data standardization, and data cleansing.
6. **Data exploration:** Users can examine data stored in Hadoop using Hive, which enables them to spot patterns and trends in huge datasets.
7. **Data Transformation:** Data mapping, aggregation, and filtering are all supported by Hive.

2. What are the different types of tables available in Hive?

Hive supports several types of tables for storing and managing data:

1. **Managed Tables:** Hive manages these tables, and the data is stored in the Hive warehouse directory. Any data loaded into a managed table is managed by Hive automatically and can be queried directly.
2. **External Tables:** External tables are created with data from sources other than the Hive warehouse directory. The underlying data is not deleted when an external table is dropped. External tables are useful when data is generated by external systems or when other applications need to access it.
3. **Partitioned Tables:** These tables are divided into sections by one or more columns. Partitioning aids query performance by reducing the amount of data that must be scanned.
4. **Bucketed Tables:** Bucketing is a query performance improvement technique that divides large tables into smaller, more manageable pieces. Similar to partitioning, bucketing divides data into smaller, more evenly-sized groups.
5. **Virtual Tables:** Hive supports the creation of virtual tables with the HiveQL query language. Virtual tables do not store data on disc, but rather provide a virtual representation of one or more existing tables.
6. **Temporary tables:** they are created and managed within the context of a Hive session. When the session ends, these tables are automatically deleted.

Each type of table in Hive has its own advantages and use cases, and selecting the appropriate type of table is dependent on the application's specific requirements.

3. What is the difference between managed and external tables?

The main distinction between external and managed tables in Hive is the location of the table data.

Sr. No.	Managed Tables	External Tables
1.	Managed tables are also known as internal tables, and Hive manages the data for these tables within the Hive warehouse directory. Hive creates a directory in the warehouse directory to store the table data when a managed table is created.	External tables do not use the Hive warehouse directory to store data. Instead, they refer to data stored elsewhere, such as HDFS, Amazon S3, or another external system.
2.	Hive typically creates and manages the data for a managed table, for example, it does all operations on the table, such as inserting or deleting data.	Hive only creates a metadata definition for an external table when it is created, such as the schema and location of the data. Hive does not manage the underlying data, which can be created and managed by any external system or process.

To summarize, external tables refer to data stored outside of the Hive warehouse directory, whereas managed tables manage their data within the Hive warehouse directory.

4. Where does the data of a Hive table get stored?

The location of the data for a Hive table is determined by whether the table is managed or external.

1. The data for managed tables are stored in the Hive warehouse directory, which is usually located in HDFS (Hadoop Distributed File System). Hive creates a subdirectory within the warehouse directory to store data when a managed table is created.
2. The data for external tables are typically stored outside of the Hive warehouse directory, in an external storage system such as HDFS, Amazon S3, or another external system.

Hive only creates a metadata definition for an external table when it is created, such as the schema and location of the data. Hive does not manage the underlying data, which can be created and managed by any external system or process.

The data for a Hive table is distributed across the nodes of the Hadoop cluster in both cases, allowing for parallel processing and scalability.

5. Can Hive be used in OLTP systems?

Hive is primarily intended for use in OLAP (Online Analytical Processing) systems, which process large amounts of data for complex queries and analysis. Hive is designed for batch processing of large datasets and **is not suitable for OLTP** (Online Transaction Processing) systems that require quick and efficient processing of individual transactions.

OLTP systems are typically used for transactional processing, such as online order processing or real-time inventory management. These systems necessitate high concurrency and low latency, neither of which Hive excels at. Hive is intended for batch processing of large datasets, so it may not be appropriate for OLTP systems. Hive can be used in tandem with other systems to support OLTP use cases. Hive, for example, can be used to transform and aggregate data for OLTP systems as part of a data processing pipeline. Hive can also be used to generate reports and provide insights into OLTP data, which can aid in decision-making processes.

While Hive is not well-suited for OLTP systems, it can be used in tandem with other systems to support OLTP use cases or provide insights into OLTP data.

6. Can a table name be changed in Hive?

Yes, it is possible to rename a table in Hive using the RENAME command.

To rename a table, we can use the following syntax:

```
ALTER TABLE old_table_name RENAME TO new_table_name;
```

For example, if we have a table named "employees" and we want to rename it to "staff", we can run the following command:

```
ALTER TABLE employees RENAME TO staff;
```

Note that when we rename a table in Hive, the table metadata is updated to reflect the new table name, but the underlying data is not moved or changed. The table's location in HDFS remains the same, and any data stored in the table is still accessible using the new table name.

It is important to update any references to the table in our code or queries after renaming the table to ensure that our code continues to work correctly.

7. Can the default location of a managed table be changed in Hive?

Yes, it is possible to change the default location of a managed table in Hive by updating the value of the `hive.metastore.warehouse.dir` property in the Hive configuration file.

By default, the location of the Hive warehouse directory is set to `"/user/hive/warehouse"` in HDFS, but we can change this location to a different directory or HDFS path by following these steps:

1. Open the Hive configuration file, which is typically located at `/etc/hive/conf/hive-site.xml` on the Hadoop cluster.
2. Find the `hive.metastore.warehouse.dir` property and update its value to the new directory or HDFS path where we want to store the managed tables. For example, we can set the value to `/user/myhivedata/warehouse`.
3. Save the changes to the configuration file and restart Hive services to apply the changes.

After we update the value of `hive.metastore.warehouse.dir`, any new managed tables created in Hive will be stored in the new directory or HDFS path. Existing managed tables will continue to use their original location, so we will need to move the data for these tables to the new location manually if desired.

8. What is a Hive Metastore?

In Apache Hive, the Hive Metastore is a central data repository that contains the metadata of Hive tables, which includes their structure, partitioning, and storage location. The Hive Metastore is used by Hive clients and servers to retrieve and update metadata information about tables. It provides a unified view of data stored in various formats like HDFS, Apache HBase, and Amazon S3.

The Hive Metastore is designed to separate metadata management from data processing, making it possible to manage metadata more efficiently and flexibly across multiple Hive instances. It can be configured as a local or remote database, depending on the deployment architecture and scalability requirements of the Hive installation. By default, the Hive Metastore uses an embedded Derby database as its local megastore. However, other databases like MySQL, PostgreSQL, and Oracle can also be used as remote metastores.

9. What are the types of Metastore?

Hive supports two types of Metastore:

1. **Embedded Metastore:** The Embedded Metastore is the default Metastore that is provided with Hive. It is implemented as an embedded Derby database that is included with Hive and is used to store metadata for Hive tables. The Embedded Metastore is suitable for small-scale deployments where performance is not critical, but it is not recommended for larger or more complex metadata deployments.
2. **External Metastore:** An External Metastore is a standalone Metastore that is implemented as a separate database outside of Hive. It can be used to store metadata for multiple Hive instances, which makes it ideal for larger or more complex deployments. The External Metastore supports a variety of databases, including MySQL, PostgreSQL, Oracle, and Microsoft SQL Server.

Both types of Metastore provide similar functionality, including the ability to store metadata for Hive tables, databases, and columns, as well as support for schema evolution, metadata management, and data portability. The choice of which type of Metastore to use depends on the specific requirements of our deployment, including the size and complexity of data, performance needs, and existing database infrastructure.

10. What is the difference between Local and Remote Metastore?

The distinction between local and remote metastore in Hive is primarily related to their location and method of access. A local metastore is located on the same machine as the Hive server and contains metadata for tables and partitions. Local metastores are generally used in single-node Hive installations where the Hadoop Distributed File System (HDFS) and Hive server are on the same machine.

On the other hand, a remote metastore is located on a separate machine and is accessed over a network connection, usually via a JDBC or ODBC driver. Remote metastores are used in distributed Hive installations with multiple Hive servers and/or HDFS clusters deployed across different machines. In these scenarios, a single metastore can manage metadata for all the tables and partitions in the various Hive installations.

Choosing between a local or remote metastore depends on the deployment architecture and scalability requirements. Local metastores are simpler to set up and adequate for small-scale installations. In contrast, remote metastores provide greater scalability and flexibility for larger, distributed deployments.

11. What is the default Apache Hive metastore database?

"Derby" is the name of the default Apache Hive Metastore database. It is a Hive-included embedded database that is used by default to store metadata for Hive tables. The Derby database is a lightweight, single-user database that is appropriate for small-scale deployments and testing but not for production use or larger deployments.

It should be noted that the Derby database is not scalable and may experience performance limitations in larger deployments. For larger-scale deployments, it is therefore recommended to use an external Metastore database. MySQL, PostgreSQL, Oracle, and Microsoft SQL Server are among the external Metastore databases supported by Hive.

To use an external Metastore database, we need to configure Hive to use the external database and create the necessary tables in the database using the Hive schema tool.

12. Can multiple users use one metastore?

Yes, multiple users can use a single Metastore in Hive, that is, it allows multiple Hive servers or clients to share the same metadata, which can simplify management and improve consistency across the deployment.

When multiple users are accessing the same Metastore, it is important to ensure that the Metastore is configured to support concurrency and that appropriate access controls are in place to prevent conflicts and unauthorized access. Hive provides several configuration options for controlling concurrent access to the Metastore, including locking and transaction isolation levels. These settings can be adjusted based on the specific requirements of the deployment.

Additionally, it is recommended to use an external Metastore database for larger-scale deployments with multiple users, as this can improve scalability and provide better performance than the default embedded Metastore database. External Metastore databases can support multiple concurrent connections and can be configured with appropriate security and access controls to ensure data integrity and prevent unauthorized access.

13. What are the three different modes in which Hive can be operated?

Hive can be used in three different modes:

- **Local or Standalone Mode:** Hive runs as a single process on a single machine in Local Mode. This mode is appropriate for testing and development because it allows experimenting with Hive without the need for a cluster or additional resources.
- **Pseudo-distributed Mode:** In pseudo-distributed mode, Hive is run on a single machine, but the machine is configured to simulate a distributed environment by running multiple processes, such as NameNode, DataNode, ResourceManager, and NodeManager, on the same machine. This configuration allows Hive to take advantage of the parallel processing capabilities of Hadoop, making it useful for testing and development.
- **Fully-distributed Mode:** Distributed mode refers to the mode in which Hive is run on a cluster of machines. The data is stored in a distributed file system like HDFS, and the processing is done across multiple nodes in the cluster. Hive queries are translated into jobs that can be executed in parallel across the cluster, allowing for high performance and scalability. The three modes MapReduce, Tez, and Spark all fall under this type of distributed mode.
 - **MapReduce Mode:** In MapReduce Mode, Hive runs on a Hadoop cluster and processes queries using the Hadoop MapReduce framework. This mode is appropriate for production deployments and large-scale data processing because it allows Hive to leverage Hadoop's distributed processing capabilities.
 - **Tez Mode:** In Tez mode, Hive runs on a Hadoop cluster using the Tez execution engine, which is designed to provide high-performance data processing. Hive translates the queries into Tez tasks, which are then executed on the cluster. This mode is suitable for processing large datasets that require high-performance data processing and can benefit from the optimized execution plan provided by Tez.
 - **Spark mode:** Hive runs on a Spark cluster and processes queries using the Spark execution engine in Spark Mode. This mode is appropriate for deployments that use Spark as their primary processing framework because it enables Hive to benefit from Spark's advanced processing capabilities and performance optimizations.

14. Is there a data type in Hive to store date information?

Yes, Hive has a data type called "DATE" to store date information and also has one for Timestamps. The DATE data type represents a date value in the format of "YYYY-MM-DD". The value of date can range from January 1, 0001 to December 31, 9999.

In Hive, the DATE data type is supported for both Managed and External tables. It can be used in table schemas, as well as in SQL statements to query or manipulate data.

Some of the functions that can be used with the DATE data type in Hive include:

- **TO_DATE():** converts a string value to a DATE data type.
- **YEAR():** extracts the year value from a DATE data type.
- **MONTH():** extracts the month value from a DATE data type.
- **DAY():** extracts the day value from a DATE data type.
- **ADD_MONTHS():** adds a specified number of months to a DATE data type.
- **DATEDIFF():** calculates the number of days between two DATE data types.

In Hive, using the DATE data type can be useful for data analysis and reporting as it allows performing date-based queries and aggregations on our data.

15. Why is partitioning used in Hive?

In Hive, partitioning is utilized to enhance query performance and manage huge data sets effectively. Partitioning involves dividing data into smaller, more manageable parts based on specified column(s), allowing users to analyze specific subsets of data easily without scanning the entire table. This speeds up query performance and reduces the amount of data processed during queries, thus reducing resource consumption and improving overall system performance. Moreover, partitioning can be utilized to optimize data storage by storing data in a more efficient way based on the partitioning column(s).

16. What is dynamic partitioning and when is it used in Hive?

Dynamic partitioning is a Hive feature that allows us to automatically create partition

When we have a large amount of data that needs to be partitioned based on one or more columns, but we don't want to manually create and manage the partitions, we use dynamic partitioning. This is especially useful when dealing with unstructured data, where determining the appropriate partitioning scheme ahead of time may be difficult or time-consuming.

To use dynamic partitioning in Hive, we must first enable it by setting the following properties in the Hive configuration:

```
set hive.exec.dynamic.partition=true;  
set hive.exec.dynamic.partition.mode=nonstrict;
```

Once dynamic partitioning is enabled, we can use the INSERT OVERWRITE statement with the dynamic partitioning option to load data into a partitioned table. For example, to partition a table by date, we might use the following statement:

```
INSERT OVERWRITE TABLE mytable PARTITION (date)  
SELECT col1, col2, date FROM mydata;
```

In this example, Hive will automatically create partitions based on the values in the "date" column of the "mydata" table, and load the data into the appropriate partition in the "mytable" table. By using dynamic partitioning in this way, we can simplify the data loading process and improve query performance by partitioning the data more efficiently.

17. What are the Hive collection data types?

Hive offers support for various collection data types including:

- **Array:** an ordered list of same-type elements.
- **Map:** a set of key-value pairs with different data types for keys and values.
- **Struct:** a group of fields with their own data types
- **Union:** a collection of data types where one data type is used at a time.

Using these collection data types in Hive, users can perform more sophisticated data modelling and analysis, especially with semi-structured or nested data. These types can be used in HiveQL queries and the elements inside them can be modified and accessed using built-in functions.

18. Is it possible to run UNIX shell commands in Hive?

Yes, it is possible to run UNIX shell commands in Hive using the "!sh" or "!bash" commands. These commands allow users to execute any UNIX shell command within the Hive shell.

For example, to list the files in a directory using the "ls" command, we can use the following syntax:

```
!sh ls /path/to/directory
```

Similarly, the user can use the "!sh" command to execute any other UNIX shell command, such as "grep", "awk", or "sed". This can be useful for performing operations on data outside of Hive, such as filtering or transforming data using standard UNIX tools.

It's important to note that running shell commands within Hive can be risky, as it can potentially allow users to execute arbitrary commands on the host system. Therefore, it's generally recommended to only allow trusted users to execute shell commands in Hive and to carefully review and sanitize any commands before executing them.

19. Is it possible to execute Hive queries from a script file?

Yes, it is possible to execute Hive queries from a script file. It supports several execution modes, including an interactive shell mode and a batch mode, which allows us to run Hive queries from a script file.

To execute Hive queries from a script file, the user can create a text file containing the Hive queries one wants to execute, and then pass this file to the "hive" command using the "-f" option. For example:

```
hive -f /path/to/script/file.hql
```

This will execute all the Hive queries contained in the script file "file.hql". We can also include additional options and parameters with the "hive" command, such as specifying the name of the database or output file to use.

Using a script file to execute Hive queries can be useful for automating repetitive tasks, as well as for processing large volumes of data in a batch mode. It can also help to improve productivity and reduce errors, as we can write and test the queries in a text editor before running them in Hive.

20. What is a .hiverc file?

A ".hiverc" file is a Hive configuration file that specifies default Hive shell settings and options. Hive reads the ".hiverc" file at startup and applies any settings or options specified in the file to the Hive shell.

The ".hiverc" file is a text file that can be edited with any text editor. It can contain any valid Hive command, such as setting environment variables, specifying default Hive command options, and defining aliases for commonly used commands. The ".hiverc" file is commonly used for the following purposes:

1. Changing the default Hive options: The ".hiverc" file can be used to set default options for commonly used Hive commands, such as the default output format or delimiter for query results.
2. Creating aliases: To save time and improve productivity, we can create aliases for commonly used Hive commands. For example, we could make "SELECT * FROM table" **an** alias **of** "sel *".
3. Setting environment variables: The ".hiverc" file can be used to set environment variables used by Hive, such as the HADOOP_HOME and HIVE_CON_DIR variables.

Overall, the ".hiverc" file is a helpful tool for customizing and configuring the Hive shell to our specific requirements and preferences. When working with Hive, we can streamline our workflow and increase our productivity by defining default settings and options in the ".hiverc" file.

21. Are multi-line comments supported by Hive?

Yes, Hive supports multi-line comments using the standard SQL comment syntax of `"/ * ... */`. This allows us to add comments that span multiple lines of code and can be used to provide context or explanations for complex queries.

For example, to add a multi-line comment to a Hive query, we can use the following syntax:

```
/* This is a multi-line comment
   that spans multiple lines
   and provides context for the query */
SELECT col1, col2 FROM my_table;
```

This will execute the Hive query and ignore the text within the comment, allowing us to include detailed comments without affecting the functionality of the query.

It's important to note that comments should be used sparingly and should be relevant and helpful to other users who may need to review or modify our code. By using clear and concise comments, we can help to improve the readability and maintainability of our Hive code.

22. How can you view the indexes of a Hive table?

To view the indexes of a Hive table, you can use the following command:

```
SHOW INDEXES ON table_name;
```

Replace `"table_name"` with the name of the table whose indexes you want to view. This command will display a list of all the indexes created on the specified table along with their associated metadata, such as the index name, indexed column(s), and index table. If no indexes exist on the table, the command will return an empty result set.

23. What is the Hive ObjectInspector function?

The Hive ObjectInspector function is an important feature of the Apache Hive data warehouse software used to examine and modify the internal data structures of tables. It is a vital element in the process of executing Hive queries, as it transforms the raw data saved in Hive tables into a format that Hive operators can handle. The ObjectInspector function is responsible for creating a standard interface for Hive operators to access table data, and it supports multiple data types, including user-defined, complex, and primitive types. Hive operators extensively use this function in HiveQL queries, and it is a valuable tool for manipulating data within Hive.

24. What is Bucketing in Hive?

Bucketing is a Hive technique for dividing data in a table into more manageable and efficient sets of files. Data is divided into a fixed number of buckets using bucketing based on the value of a hash function applied to a specific column in the table. Each bucket is then stored as a separate file, allowing Hive to read and write data more efficiently based on the partitioning key.

Bucketing, like partitioning, allows Hive to optimize queries by limiting the amount of data that must be scanned. Unlike partitioning, which creates directories based on specific column values, bucketing generates fixed-size files based on a column's hash value.

Hive can reduce the amount of I/O required to read and write data and improve query performance by limiting the number of files that must be scanned by bucketing data. Bucketing is especially useful for tables that are very large or have a lot of data skew, as it helps to distribute the data more evenly and reduces the impact of hotspots.

Overall, bucketing is an effective tool for managing and optimizing large datasets in Hive, and it can be combined with other Hive features such as partitioning, indexing, and compression to improve performance and efficiency even further.

25. How is bucketing helpful?

Bucketing is a helpful technique in Hive for improving query performance and managing large datasets. Some of the ways in which bucketing can be helpful are:

1. **Reduced I/O:** Bucketing can help to reduce the amount of I/O required to read and write data by dividing it into a fixed number of files. This can help in improving query performance by limiting the number of files that need to be scanned.
2. **Data Skew:** Bucketing can help to distribute data more evenly, which can be particularly useful for tables with a high degree of data skew. By spreading the data more evenly across the buckets, bucketing can help to reduce the impact of hotspots and improve query performance.
3. **Join performance:** When tables are joined on the bucketing key, Hive can perform a map-side join operation, which can be much faster than a traditional reduce-side join operation. This is because Hive can read data directly from the bucketed files rather than having to shuffle and sort the data across the network.
4. **Sampling:** Bucketing can also be used to sample data from a table more efficiently. By selecting a subset of the buckets rather than scanning the entire table, Hive can quickly return a representative sample of the data.

Overall, bucketing can help to improve query performance, reduce the impact of data skew, and provide more efficient sampling and join operations.

26. Can you specify the name of the table creator in Hive?

Yes, it is possible to specify the name of the table creator in Hive by using the optional "COMMENT" clause when creating the table.

An example is as follows:

```
CREATE TABLE my_table (  
    col1 string,  
    col2 int  
)  
COMMENT 'Created by Victor'  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ',';
```

In this example, the table "my_table" is created with two columns, "col1" and "col2". The "COMMENT" clause is used to specify that the table was created by "Victor". The remaining lines of the query specify the format of the data in the table.

When we run the "DESCRIBE FORMATTED my_table" command, the output will include the table creator name in the "Table Parameters" section:

Table Parameters:

```
comment Created by Victor
```

```
...
```

By specifying the name of the table creator, we can add important metadata to the tables and help to track who created them and when. This can be particularly useful in a collaborative environment where multiple users are working with the same data.

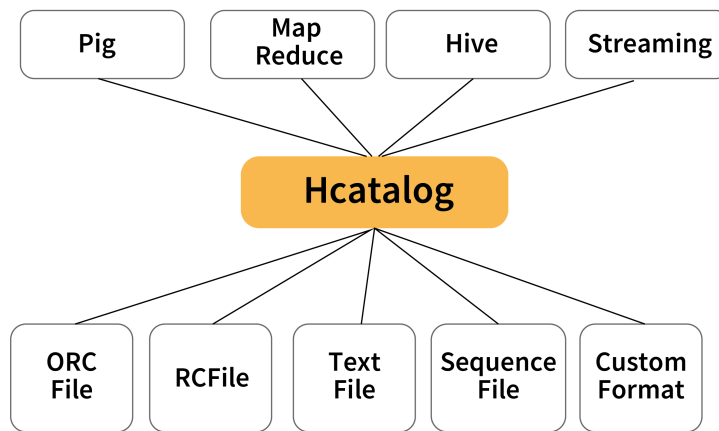
27. What is HCatalog?

HCatalog is an Apache Hive project component that serves as a storage and table management layer for the Hadoop clusters.

HCatalog is a tool for accessing Hive metastore tables from Pig, Spark SQL, and/or custom MapReduce applications. We can create tables and perform other operations with HCatalog's REST interface and command line client. We then write our applications to access the tables using HCatalog libraries. It offers a consolidated schema and metadata model for data in Hadoop. It enables the users to manage and access the data using multiple programming languages and tools.



Hcatalog Architecture



With the HCatalog, it becomes easier to do the creating, modifying, and deleting the tables in Hadoop as it stores metadata about the tables in one central location.

Some of the important key features of the HCatalog are as follows:

1. Table management
2. Schema management
3. Integration with other tools
4. Data access
5. Security
6. Data type mapping

28. What is UDF in Hive?

“UDF” stands for a User-Defined Function, and refers to a custom function written by the user to perform a specific task on the data in Hive. Hive has a number of built-in functions for manipulating and processing data. In some cases, these functions may not be sufficient to meet the needs of a specific use case. In such cases, users can create their own custom Hive functions in Java or any other programming language supported by Hive.

Hive UDFs can be scalar, aggregate, or table functions, and they can take one or more input arguments and return a single value or a collection of values. Some common examples of UDFs in Hive include functions to parse, extract, and transform data, perform custom aggregations, and implement complex business logic.

Users must first write the function code and then compile it into a JAR file in order to create a UDF in Hive. They can then use the CREATE FUNCTION statement to register the function in Hive, providing information such as the function's name, input arguments, and return type. The UDF, once registered, can be used in Hive queries just like any other built-in function. UDFs in Hive can be a powerful tool for data processing and analysis because they enable users to implement custom logic and manipulate data in ways that built-in functions alone cannot.

It is important to note that UDFs can cause performance issues if they are not written efficiently or used excessively in queries. As a result, it is critical to carefully assess the need for a UDF and optimize its implementation for maximum performance.

29. What does `/*streamtable(table_name)*/` do?

`/*streamtable(table_name)*/` is a special comment syntax in Hive that allows users to indicate that a particular table should be treated as a stream table. A stream table is a table that is continuously updated with new data, such as a stream of events or messages. By designating a table as a stream table using this comment syntax, Hive can automatically detect and process new data as it becomes available, rather than requiring users to manually update the table.

When a table is marked as a stream table using the `/*streamtable(table_name)*/` syntax, Hive sets up a continuous processing pipeline that can handle data in real time. This pipeline typically involves a combination of tools and technologies, such as Flume or Kafka for data ingestion, Spark Streaming or Storm for real-time processing, and Hadoop or HBase for storage and retrieval.

Using a stream table in Hive can be useful for a wide range of use cases, such as monitoring and analyzing log data, processing social media feeds, or handling real-time financial transactions. However, it is important to note that setting up and maintaining a stream table in Hive can be complex and requires careful planning and coordination between different tools and technologies.

Hive Interview Questions for Experienced

30. What are the key features of Hive that differentiate it from other big data processing tools?

Some of Hive's key features that set it apart from other big data processing tools:

1. **SQL-like syntax:** It supports a SQL-like language that many users are familiar with, making it simple for people with SQL experience to work with big data.
2. **Flexible:** It employs a Schema on Read approach, which means that data can be ingested into Hive without a predefined schema. This allows for greater flexibility and agility in data processing.
3. **Compatible:** It supports a variety of storage formats, including text, CSV, JSON, ORC, etc. making it simple to work with various types of data.
4. **Integration with the Hadoop ecosystem:** It works well with other Hadoop ecosystem tools such as HDFS, MapReduce, etc. allowing users to work with big data in a distributed environment.
5. **Customizable:** It allows users to create custom functions and scripts in Java, Python, or other programming languages that can be incorporated into Hive queries.
6. **Optimizations:** It includes a number of built-in optimizations, such as query optimization and indexing, that can improve performance and decrease processing time.
7. **User-defined functions (UDFs):** It enables users to create and use custom UDFs to extend Hive's functionality, making complex data processing tasks easier.

Overall, Hive's SQL-like language, support for different storage formats, integration with the Hadoop ecosystem, customizability, and optimizations make it a powerful tool for processing and analyzing big data.

31. Can you walk me through the Hive query optimization process and the techniques you use to optimize queries?

The Hive query optimization process and some techniques to optimize queries are as follows:

1. **Analyze query performance:** Analyzing query performance is the first step in optimizing a Hive query. To obtain information about the query execution plan and identify areas where the query can be optimized, we can use Hive's built-in EXPLAIN command.
2. **Use appropriate file formats:** The file format used can have a significant impact on query performance. Hive supports several file formats that are optimized for query performance, including ORC, Parquet, and Avro.
3. Hive supports partitioning and bucketing, which can improve query performance by decreasing the amount of data scanned.
4. Hive supports a variety of join techniques, including map-side joins, broadcast joins, etc. By reducing data movement, the right join technique can improve query performance.
5. **Use appropriate data types:** The use of appropriate data types can also affect query performance. For example: using integer data types instead of string data types can improve query performance.
6. **Optimize data skew:** In Hive queries, data skew can cause significant performance issues. One method for reducing data skew is to estimate the distribution of data and then adjust the query plan accordingly.
7. **Tune configuration parameters:** Memory allocation, parallelism, and compression settings are just a few of the configuration parameters that can be tweaked to improve query performance.

32. What is the difference between partitioning and bucketing in Hive? When would you choose one over the other?

Partitioning and bucketing are the two techniques used in Hive to improve query performance by organizing data in a more efficient way.

Sr. No	Partitioning	Bucketing
1.	Partitioning is a way of dividing a large table into smaller, more manageable parts based on the values of one or more columns. The partition key is used to create a directory hierarchy that represents the partitioning scheme, and each partition is stored as a separate directory with its own set of files.	Bucketing is a way of dividing data within a partition into more manageable chunks called buckets, based on the hash value of one or more columns.
2.	Partitioning is useful when we need to filter data based on the partition key, as Hive can quickly skip over partitions that don't match the filter condition.	Bucketing is useful when we need to join or aggregate data on a specific column, as it can help reduce the number of data files that need to be scanned.

It is also possible to use both techniques together to create a more fine-grained data organization scheme. The decision to use partitioning, bucketing or both depends on the specific requirements of the use case and the characteristics of the data being processed.

33. Can you give an example of a complex Hive query you have written and the problem it solved?

Consider a scenario where we have a large dataset containing customer transactions, and we want to find the top 10 customers based on the total amount of money they spent. Here's a complex Hive query that can solve this problem:

```
SELECT customer_id, SUM(amount) AS total_spent
FROM transactions
GROUP BY customer_id
ORDER BY total_spent DESC
LIMIT 10;
```

In this query, we're using the SUM function to calculate the total amount spent by each customer, grouping the results by customer_id. We then sort the results in descending order based on the total_spent column and use the LIMIT clause to return only the top 10 results.

This query is complex because it involves grouping, aggregating, sorting, and limiting the results, all of which can be resource-intensive operations on a large dataset. By using Hive, we can take advantage of its distributed processing capabilities to perform these operations in parallel across a cluster of machines, enabling us to process large datasets quickly and efficiently.

34. How do you handle data skewness in Hive? How have you used Hive in conjunction with other big data tools like Spark or Hadoop?

The handling of data skewness in Hive is critical because it can have a significant impact on query performance. Bucketing is a common technique for dealing with data skewness, which involves dividing the data into smaller, evenly sized buckets based on a key column. This can help distribute data more evenly across cluster nodes, improving query performance. Another approach is to use dynamic partitioning, which involves partitioning data on the fly based on query conditions. By dividing the data into smaller partitions based on the query conditions, can help reduce the impact of data skewness. There are numerous applications for using Hive in conjunction with other big data tools such as Spark or Hadoop.

We could, **for example**, use Spark to pre-process the data before loading it into Hive for querying, or we could use Hadoop for distributed storage and Hive for querying. Hive could also be used as part of a larger data processing pipeline that includes a variety of tools and technologies. The key is to select the appropriate tool for each stage of the pipeline based on its strengths and limitations and to ensure that the various tools can coexist seamlessly.

35. Can you explain how dynamic partitioning works in Hive?

Hive's dynamic partitioning allows us to partition data on the fly based on the values in one or more columns in a table. It comes in handy when we need to divide a large dataset into smaller, more manageable chunks based on some criteria.

Dynamic partitioning works by allowing us to specify a partitioning column in our query, which is then used to generate new partitions as needed. Hive examines the values in the partitioning column and creates new partitions for each unique value it finds when we run a query that uses dynamic partitioning.

For example, suppose we have a large table of customer orders that we want to partition by date. The table has columns for the customer ID, order ID, order date, and other order details. To partition the table by date, we could use the following command:

```
INSERT INTO TABLE orders PARTITION (order_date)
SELECT customer_id, order_id, order_date, order_details
FROM orders_raw;
```

This command inserts orders_raw table data into the orders table, partitioned by order_date. For each unique order date value found in the orders_raw table, Hive creates a new partition.

Dynamic partitioning is especially useful when we have a large dataset that cannot be partitioned in advance, or when we want to partition the data based on an unknown column. It is, however, slower than static partitioning because Hive must scan the entire dataset to determine the partition values.

36. Have you worked with Hive UDFs (user-defined functions) or UDAFs (user-defined aggregate functions)?

Certainly, I have experience working with UDFs and UDAFs. UDFs stand for user-defined functions, while UDAFs stand for user-defined aggregate functions. These functions are created by the user to perform customized operations on data within a Hive query. UDFs process only one row at a time and return a single row as output. The UDAFs operate on multiple rows and return a single output row and aggregate functions which include functions such as COUNT and MAX. e.g. Hive's built-in COUNT() or MAX() functions. Both UDFs and UDAFs can be written in different programming languages such as Java, Python, and more. They can be utilized within Hive queries to manipulate and process data.

37. What are some best practices for managing and optimizing Hive metastore performance?

Some of the best practices for managing and optimizing **Hive metastore** performance:

1. **Choose the right type of metastore database:** Metastore databases are classified into two types: embedded and remote. We can use the embedded metastore database if we have a small or medium-sized cluster. If we have a large cluster, however, we should use a remote metastore database such as MySQL or PostgreSQL.
2. **Adjust the database parameters:** Tuning database parameters such as memory allocation, caching, and database size can improve the performance of the metastore database.
3. **Divide the table:** Partitioning can improve query performance significantly by reducing the amount of data that must be scanned. In the WHERE clause, we should partition the tables based on the most frequently used columns.
4. **Bucket the tables:** Bucketing can help to evenly distribute data across nodes, improving query performance. The bucketing columns should be chosen based on the join conditions or grouping clauses in our queries.
5. **Use appropriate file formats:** The file format we use can have a big impact on query performance. The ORC file format, for example, is designed for large-scale data processing and can improve query performance.
6. **JVM monitoring and tuning:** Hive metastore is run on the Java Virtual Machine (JVM), and it is critical to monitor and tune the JVM parameters for optimal performance.
7. **Maintain the metastore database:** It is critical to keep the metastore database in good working order. We should compact and optimize the database on a regular basis to remove unnecessary data and improve performance.

We can effectively manage and optimize Hive metastore performance by following these best practices, which can help to improve the overall performance of our Hive queries.

38. Can you explain how Hive authorization and authentication work?

Yes, I can explain how Hive authorization and authentication work.

To control access to Hive resources and data, authorization and authentication are used. Authorization entails defining policies that govern who can and cannot perform specific actions on Hive resources such as tables or databases. Authentication is the process of verifying a user's or process's identity. Hive supports the following authorization and authentication mechanisms:

1. **SQL authorization based on standards:** This mechanism is based on SQL standards and provides an easy way to manage Hive users and roles.
2. **Apache Ranger:** Apache Ranger is a free and open-source framework for centralized authorization and audit management in Apache Hadoop ecosystems. It allows for fine-grained access control policies, such as column-level access control.
3. **Apache Sentry:** Apache Sentry is another open-source framework that provides role-based access control to Hive resources. It allows administrators to define roles and privileges for specific databases and tables.
4. **LDAP/AD integration:** Hive can integrate with LDAP or Active Directory (AD) to authenticate users.

In addition to these mechanisms, Hive also supports Kerberos authentication, which is a network authentication protocol that provides strong encryption of user credentials.

To optimize Hive authorization and authentication performance, it is important to follow some best practices, such as:

1. Use SQL standards-based authorization for simple use cases.
2. Use Apache Ranger or Apache Sentry for fine-grained access control.
3. Use LDAP or AD integration for authentication.
4. Use Kerberos authentication for secure network authentication.
5. Regularly review and update access control policies to ensure they remain relevant and effective.

39. How would you address the issue of creating a single Hive table for multiple small CSV files located in the /input directory of HDFS, without compromising the system's performance, given that using many small files can slow down Hadoop's performance?

The CSV files contain data in the following format: {id, name, e-mail, country}.

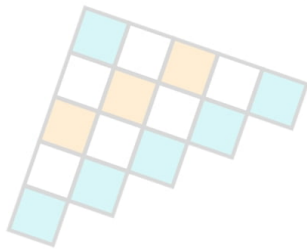
There are various methods to address the issue and enhance the system's efficiency:

- **Merge the small CSV files into bigger files:** To decrease the quantity of input files and enhance the overall performance of the system, we can combine the small CSV files into larger files using Hadoop's FileUtil class or the command ``hadoop fs -getmerge`` before constructing the Hive table.
- **Take advantage of Hive's external table feature:** Instead of creating a managed table in Hive, you can establish an external table that directly references the small CSV files in HDFS. This can enhance the system's performance by avoiding the overhead of transferring data into Hive's managed tables.
- **Utilize Hive's partitioning feature:** If the CSV files are partitioned based on a particular criterion, such as date, you can use Hive's partitioning feature to create a partitioned Hive table. This approach can optimize query performance by enabling Hive to skip irrelevant partitions and avoid scanning the entire data set, ultimately resulting in improved query performance.
- **Use a different storage format:** If the CSV files are not compressed, you may consider utilizing a compressed format like ORC or Parquet. These formats can minimize the storage space required for data and enhance query performance.
- **Use Hadoop's SequenceFile format:** Hadoop's SequenceFile format is a binary file format that can be used to store large chunks of data. We can transform the small CSV files into a SequenceFile format using Hadoop's SequenceFile.writer class, and then on top of the sequence files, establish a Hive table. With these reduced numbers of small files, we can improve the overall performance of the system.

Conclusion

In summary, Hive has grown in popularity due to its ability to manage and analyze large datasets stored in the Hadoop Distributed File System via a SQL-like query language. Because of its scalability and SQL-like syntax, it is usable by SQL-experienced users. Candidates should be familiar with Hive's architecture, data types, query language, and optimization techniques before attending an interview. By evaluating the candidate's knowledge and skills in Hive, interviewers can assess the candidate's ability to work with large datasets and perform complex data analysis tasks. A solid understanding of Hive can be extremely beneficial to data professionals pursuing careers in data management, data analysis, or business intelligence. It is an essential technology for these industries to understand.

Read more here: <https://hive.apache.org/>



Links to More Interview Questions

[C Interview Questions](#)

[Php Interview Questions](#)

[C Sharp Interview Questions](#)

[Web Api Interview Questions](#)

[Hibernate Interview Questions](#)

[Node Js Interview Questions](#)

[Cpp Interview Questions](#)

[Oops Interview Questions](#)

[Devops Interview Questions](#)

[Machine Learning Interview Questions](#)

[Docker Interview Questions](#)

[Mysql Interview Questions](#)

[Css Interview Questions](#)

[Laravel Interview Questions](#)

[Asp Net Interview Questions](#)

[Django Interview Questions](#)

[Dot Net Interview Questions](#)

[Kubernetes Interview Questions](#)

[Operating System Interview Questions](#)

[React Native Interview Questions](#)

[Aws Interview Questions](#)

[Git Interview Questions](#)

[Java 8 Interview Questions](#)

[Mongodb Interview Questions](#)

[Dbms Interview Questions](#)

[Spring Boot Interview Questions](#)

[Power Bi Interview Questions](#)

[Pl Sql Interview Questions](#)

[Tableau Interview Questions](#)

[Linux Interview Questions](#)

[Ansible Interview Questions](#)

[Java Interview Questions](#)

[Jenkins Interview Questions](#)